

1 Topics every Cybersecurity Engineer must know in depth

1. Networking & TCP/IP

Acting as a PhD in information technology with over 20 years of experience teaching cybersecurity, help me learn the following Networking & TCP/IP concepts:

- 1 - Understand how data moves across networks at layers 2–7: Ethernet, VLANs, ARP, IP, ICMP, TCP/UDP, routing, DNS, HTTP(S), and common application protocols.
- 2 - Get deep knowledge of ports, connection states, MTU, fragmentation, NAT, and load balancers is critical to correctly design controls and debug security issues.
- 3 - How to read and interpret packet captures, distinguish normal versus anomalous flows, and reason about how attacks like spoofing, scanning, and session hijacking actually traverse the network.
- 4 - Subnetting and CIDR, routing protocols basics (OSPF/BGP concepts), and how virtual networks (VPCs, VNet, overlays) map onto physical networking.
- 5 - Firewall rules, IDS/IPS tuning, and incident triage at the network layer.

[linkedin](#)

2. Operating Systems & System Hardening (Linux/Windows)

You should understand the architecture of at least Linux and Windows: process model, user/kernel space, memory management, filesystem permissions, services/daemons, logging, and startup mechanisms. System hardening means reducing attack surface by disabling unnecessary services, enforcing least privilege, configuring secure logging, patching, and setting secure defaults for authentication and remote access. On Linux, this includes users/groups, sudo, PAM, SSH hardening, mandatory access control (SELinux/AppArmor), and auditing (auditd, journald). On Windows, it includes Group Policy, Windows Defender features, PowerShell security, local vs domain accounts, and event logs. You must be able to investigate suspicious processes, persistence mechanisms, and privilege escalation paths on hosts.

[chartercollege](#)

3. Identity and Access Management (IAM)

IAM is how you control who can do what, where, and when. You need deep understanding of identities (users, service accounts, workloads), authentication (passwords, MFA, certificates, tokens), and authorization models (RBAC, ABAC, least privilege). At protocol level, know SAML, OAuth 2.0, OpenID Connect, Kerberos basics, and how SSO works in web and enterprise environments. In cloud, understand provider-specific IAM constructs (roles, policies, trust relationships, conditions) and the shared responsibility aspects of identity security. In the enterprise, you should understand directory services (e.g., Active Directory concepts), federation, lifecycle management (joiner/mover/leaver), and privilege management (PIM/PAM). A Cybersecurity Engineer should be able to design secure access patterns and review access for over-privilege and toxic combinations. [linkedin](#)

4. Cryptography in Practice

Engineers must know how crypto is used, not just theory. Understand symmetric versus asymmetric encryption, key sizes, and common algorithms (AES, RSA, ECC, ChaCha20) conceptually. Know what hashing and MACs are for (integrity, password storage, message authentication) and common functions (SHA-2/3, bcrypt/Argon2). Be comfortable with TLS/SSL concepts: certificates, PKI, certificate authorities, trust chains, and typical misconfigurations (weak ciphers, protocol downgrades, certificate validation errors). In practice, you must be able to choose appropriate primitives from libraries, avoid creating custom crypto, understand key management (generation, storage, rotation), and identify weak configurations in scans and code reviews. [chartercollege](#)

5. Secure Network & Infrastructure Design

This is about architecting systems to be defensible. You should understand segmentation (VLANs, subnets, security zones), DMZ patterns, zero-trust principles (never trust, always verify), and secure remote access (VPNs, ZTNA). Know how to design firewall policies, security groups, and routing so that exposure is minimized while business needs are met. For infrastructure, this includes securing DNS, DHCP, proxies, load balancers, and reverse proxies; implementing network-based controls (NIDS/NIPS, web proxies) and understanding where to place them. In modern environments, add micro-segmentation, service meshes, and container networking security. You should be able to read an architecture diagram and identify high-risk trust boundaries and missing controls. [linkedin](#)

6. Application Security & OWASP Top 10

A Cybersecurity Engineer must understand how applications are built and how they are broken. You should know web app fundamentals (HTTP, sessions, cookies, same-origin policy, APIs/REST/JSON) and be familiar with the OWASP Top 10 classes: injection, broken auth, sensitive data exposure, insecure design, security misconfiguration, vulnerable components, identification and auth failures, access control issues, logging/monitoring failures, and SSRF. Practically, you need to interpret results from SAST/DAST, perform basic manual testing, and collaborate with developers to fix issues. You must understand secure SDLC concepts, threat modeling at feature level, and common hardening measures like input validation, output encoding, parameterized queries, proper session management, and secure error handling. [chartercollege](#)

7. Threats, Vulnerabilities, and Exploitation Fundamentals

You must deeply understand how adversaries operate. This includes malware basics (types, lifecycle), social engineering, phishing, ransomware, DDoS, credential theft, lateral movement, and data exfiltration patterns. Know what CVEs are, how vulnerability scoring (CVSS) works, and how to read advisories. Be familiar with common exploit techniques (buffer overflows, privilege escalation, token abuse, misconfiguration abuse) conceptually, even if you don't write exploits. You should be able to trace how an initial foothold can turn into full compromise using kill-chain or ATT&CK-like thinking, and use that to prioritize mitigations and monitoring. [proofpoint](#)

8. Security Operations, Monitoring & SIEM

Security operations focuses on detecting and responding to malicious activity. You need to understand logging across systems (OS, network, applications, cloud services) and how they feed SIEM/SOAR platforms. You should know what good versus suspicious looks like in logs: login anomalies, privilege changes, new services, network scans, and data exfiltration indicators. You must be able to define and tune detection rules, handle alert triage, and avoid both alert fatigue and blind spots. Knowledge of threat intelligence basics and use of indicators of compromise (IOCs) in detection workflows is key. Even if you are not a full-time SOC analyst, you must design systems with observability and actionable logs in mind. [coursera](#)

9. Incident Response & Digital Forensics Basics

Incident response (IR) is how an organization reacts to suspected compromise. You should understand the IR lifecycle: preparation, detection, analysis, containment, eradication, recovery,

and lessons learned. Engineers must help define runbooks, escalation criteria, communication paths, and evidence handling (chain of custody) so investigations are defensible. At a technical level, you should be able to acquire and preserve relevant logs, system images, and volatile data; perform structured triage; and support forensic specialists with context and hypotheses. You also need to understand the business impact of downtime and the trade-offs between aggressive containment and maintaining evidence. [coursera](#)

10. Governance, Risk, Compliance & Security Management

Technical controls must align with business, law, and regulation. You should understand basic risk management concepts: assets, threats, vulnerabilities, likelihood, impact, and controls. Know key frameworks and standards used in practice (e.g., NIST CSF, ISO 27001, CIS Controls, sector regulations like PCI DSS or data protection laws like GDPR) at a conceptual level. Cybersecurity management includes policies, security awareness, vendor management, and metrics/measurement to show control effectiveness. As an engineer, you must translate requirements into concrete designs and configurations, document controls, and contribute to audits and assessments. [coursera](#)

2 Topics to know partially (working familiarity)

11. Cloud Security (AWS, Azure, GCP)

You should understand the shared responsibility model, basic cloud IAM constructs, network security primitives (VPCs, security groups, NACLs), and common managed services (object storage, databases, serverless). Partial depth means you can review cloud architectures, identify obvious misconfigurations (public buckets, overly permissive roles, exposed management endpoints), and collaborate effectively with cloud engineers. Familiarity with cloud-native security tooling (CSPM, CWPP) and logging (CloudTrail-like audit logs) is important. [linkedin](#)

12. Container & Orchestrator Security (Docker, Kubernetes)

Containers change isolation and deployment patterns. You should know what images, registries, runtimes, and orchestrators are, and where risks appear: vulnerable base images, leaked

secrets, privilege in containers, host escape, and insecure network policies. Working familiarity means you can interpret container security scan results, understand basic Kubernetes concepts (pods, deployments, namespaces, RBAC, network policies), and recommend common mitigations like non-root containers, minimal images, and proper secret management. Deep expertise is useful but not required for every engineer. [chartercollege](#)

13. DevSecOps & CI/CD Security

DevSecOps integrates security into pipelines. You should understand the stages of CI/CD (build, test, deploy) and where to plug in SAST, DAST, dependency scanning, IaC scanning, and secret detection. Partial knowledge allows you to propose pipeline controls, interpret findings, and avoid blocking development unnecessarily. Key ideas include shift-left testing, security as code, and ensuring pipelines themselves are secured (credential storage, runner security, supply chain integrity) following modern guidance. [chartercollege](#)

14. Vulnerability Management & Patch Governance

This is the structured process of discovering, prioritizing, and remediating vulnerabilities. You should know how scanners work conceptually, the difference between authenticated and unauthenticated scans, and common challenges (false positives, coverage gaps, legacy systems). Partial mastery means you can help define SLAs, prioritize based on asset criticality and exploitability rather than CVSS alone, and coordinate with system owners. Understanding how to track remediation and communicate risk to management is key. [reddit](#)

15. Endpoint Detection & Response (EDR/XDR)

Modern defense relies heavily on EDR/XDR agents. You should know what telemetry they collect (processes, file changes, network connections), what kinds of detections they provide (behavioral, signature, ML-assisted), and how engineers integrate them into architectures. Working familiarity means you can recommend deployment coverage, assist in policy tuning (blocking vs alerting), and understand how these tools support IR and threat hunting, without needing to be the tool administrator. [coursera](#)

16. Security Awareness & Human Factors

Many attacks exploit people rather than systems. You should understand social engineering basics, phishing, business email compromise, and insider risks. Partial knowledge means collaborating with training teams to shape realistic scenarios, reinforcing secure behaviors (MFA

usage, reporting suspicious emails, handling sensitive data), and considering human usability when designing controls. Recognizing that culture, communication, and incentives often determine whether technical controls succeed is crucial. [stjohns](#)

17. Data Protection & Privacy Engineering

Data classification, data lifecycle, and privacy requirements affect design. You should know what constitutes sensitive data in your context, how it is stored, processed, and transmitted, and common controls (encryption at rest/in transit, tokenization, DLP, access monitoring). Partial familiarity with privacy principles (data minimization, purpose limitation, consent) allows you to support compliance with regulations such as GDPR-like regimes by design. Engineers help ensure architectures limit data exposure and provide traceability. [linkedin](#)

18. Security Architecture & Threat Modeling

Architecture is the “big picture” of how systems connect. You should be able to participate in architecture reviews, identify trust boundaries, and use simple threat modeling methods (e.g., STRIDE-like thinking or attacker stories) to discuss risks with teams. Partial knowledge means you can sketch data flows, identify high-value assets and likely attack paths, and propose layered controls, while more specialized security architects own standardized patterns and reference designs. [coursera](#)

19. Soft Skills: Communication, Documentation, Stakeholder Management

Security work is highly collaborative. You should be able to explain technical risks in business-relevant language, write clear incident and design documentation, and negotiate trade-offs with developers, operations, legal, and leadership. Working familiarity includes running security reviews, presenting risk assessments, and participating in cross-functional projects without being a program manager by trade. Strong communication is repeatedly highlighted as essential in cybersecurity roles. [cyber.gc](#)

20. Metrics, Security Program Management & Continuous Improvement

Security programs must be measured and improved. You should understand what makes a useful metric (leading vs lagging, actionable vs vanity) and be able to propose simple measures:

patch latency, MFA coverage, mean time to detect/respond, phishing simulation results, etc. Partial knowledge of program management helps you align engineering work with roadmaps, budget, and compliance cycles, and continuously refine controls based on incidents and new threats. [stjohns](#)

3 Topics to at least know exist

21. Red Teaming, Adversary Emulation & Purple Teaming

Beyond traditional penetration testing, red teaming focuses on full-scope, stealthy simulations of real-world adversaries, while purple teaming emphasizes collaboration between offense and defense. As an engineer, you should know these practices exist, what they aim to test (people, processes, technology), and how their findings can feed into architecture changes and detection engineering.

22. OT/ICS and SCADA Security

Operational technology in manufacturing, utilities, and critical infrastructure uses specialized protocols and systems with different safety and availability requirements than IT networks. You should be aware that ICS/SCADA security has unique constraints (legacy systems, safety over confidentiality) and reference models, and may require distinct tools, standards, and processes.

23. Hardware, Firmware, and Supply Chain Security

Security is not only software. There are risks at the hardware and firmware level (BIOS/UEFI, device firmware, embedded controllers) and in supply chains (tampered components, compromised build systems, dependency poisoning). Knowing this domain exists helps you appreciate attacks that bypass OS-level controls and the need for secure procurement, firmware management, and SBOM-style transparency.

24. Quantum-Resistant Cryptography

Cryptographic standards are evolving in response to potential quantum computing threats. You should know that post-quantum cryptography aims to replace or augment algorithms like RSA

and ECC, and that standardization efforts are ongoing. This awareness will matter for long-lived data and future migrations of protocols and key infrastructures.

25. Privacy-Enhancing Technologies (PETs)

Beyond basic privacy practices, there is a growing field of techniques such as differential privacy, homomorphic encryption, secure multi-party computation, and federated learning. Awareness of PETs helps you recognize when there might be advanced ways to analyze or share data while limiting exposure of personal information.

26. Threat Intelligence Platforms and CTI Tradecraft

Threat intelligence is more than feed ingestion. There is an entire practice around collection, analysis, and dissemination of intelligence (tactics, techniques, procedures; indicators; strategic reports) often supported by platforms and sharing communities. Knowing this exists helps you understand where high-quality detection content and context about adversaries originate.

27. Cybercrime, Law Enforcement, and Digital Evidence Handling

Cybersecurity intersects with criminal law, law enforcement cooperation, and evidentiary standards. Specialized teams handle digital evidence according to legal requirements, work with regulators, and support investigations and prosecutions. Awareness of this space ensures you escalate sensitive incidents appropriately and avoid compromising legal processes.

28. Security in Emerging Technologies (IoT, 5G, AI/ML)

Each wave of technology—IoT devices, 5G networks, AI/ML systems—introduces new architectures and threat models. Knowing that dedicated security practices exist around these domains helps you ask the right questions when your organization adopts them, even if you are not a deep expert in each.

29. Formal Methods, Secure Coding Standards, and Software Assurance

Some environments use formal verification, code contracts, and strict coding standards for high-assurance systems (e.g., safety-critical or defense). Awareness that these methods exist helps you understand why certain domains have much stricter development and review requirements than typical enterprise software.

30. Business Continuity, Disaster Recovery, and Cyber Resilience Frameworks

Beyond incident response, organizations plan for large-scale disruptions via business continuity (BC) and disaster recovery (DR) planning, sometimes organized under broader resilience frameworks. Knowing this discipline exists helps you align security engineering with backup strategy, recovery objectives, and resilient design patterns.

If you'd like, I can next take any specific topic (for example, "Identity and Access Management" or "Application Security & OWASP Top 10") and turn it into a full, standalone 1–2 page document with diagrams and example scenarios tailored to your background as a senior engineer.